

Rail Madad AI - Complaint Management System

Technical Stack Documentation

Project Overview

Rail Madad AI is an advanced complaint management system designed specifically for Indian Railways. It leverages AI-powered classification, role-based access, and real-time tracking to provide a scalable, user-friendly solution for passengers, staff, and administrators to manage railway complaints efficiently.

Technical Stack

1. Frontend Technologies

- **Framework:** Next.js 14 (App Router)
 - React-based framework optimized for server-side rendering and static site generation
 - Provides excellent performance and SEO optimization
 - Built-in routing and API routes for full-stack development
- **Language:** TypeScript
 - Provides static typing for improved reliability and developer experience
 - Reduces runtime errors and improves code maintainability
 - Enhanced IDE support with better autocomplete and refactoring
- **Styling:** TailwindCSS (v3.4) and TailwindCSS Animate
 - Utility-first CSS framework for fast, customizable UI development
 - Built-in animations and responsive design utilities
 - Consistent design system across the application
- **UI Components:** ShadCN/UI (built on Radix UI primitives)
 - Accessible and customizable React UI components
 - Consistent design language with railway-themed customizations
 - Built-in accessibility features and keyboard navigation
- **State Management:** Zustand
 - Lightweight, performant state management library
 - Simple API with minimal boilerplate
 - TypeScript-first with excellent type inference

2. Backend Technologies

- **API Layer:** Next.js API Routes
 - Built-in backend routing for creating API endpoints
 - Seamless integration with frontend components
 - Support for middleware and request/response handling
- **Database ORM:** Drizzle ORM
 - Type-safe ORM for interacting with SQL databases
 - Excellent TypeScript integration
 - Migration support and schema validation
- **Database:** PostgreSQL (Neon)
 - Managed cloud PostgreSQL database service
 - Supporting complex relational data structures
 - High availability and automatic backups
 - Scalable performance for growing user base

3. Authentication & Security

- **Authentication:** JWT-based authentication
 - Tokens created using `jsonwebtoken` library
 - Secure session management with HTTP-only cookies
 - Session-based expiration requiring manual login
- **Password Security:** bcryptjs
 - Industry-standard password hashing
 - Salt rounds for enhanced security
 - Protection against rainbow table attacks
- **Authorization:** Role-based access control
 - Separate access levels for passengers, staff, and admins
 - Granular permissions for different system features
 - Secure route protection and API endpoint access
- **Input Validation:** Zod schema validation
 - Robust API request validation

- Type-safe validation with TypeScript integration
- Client and server-side validation consistency
- **Security Features:**
 - CSRF protection for form submissions
 - SQL Injection prevention via ORM's prepared queries
 - XSS protection through proper sanitization and escaping
 - Secure cookie handling with appropriate flags

4. AI and Machine Learning

- **AI Integration:** Google Gemini AI
 - Intelligent complaint classification and categorization
 - Sentiment analysis for priority determination
 - Chatbot functionalities for user assistance
 - Natural language processing for complaint text analysis
- **AI Libraries:** @google/generative-ai
 - Official Google library for Gemini AI integration
 - Structured communication with AI models
 - Error handling and rate limiting support
- **AI Feedback Loop:**
 - Mechanism for users and admins to provide feedback
 - Continuous improvement of AI model accuracy
 - Performance metrics tracking and analysis

5. Development & Build Tools

- **Runtime:** Node.js (v18+)
 - JavaScript runtime environment
 - Excellent ecosystem and package availability
 - Long-term support and stability
- **Package Manager:** npm
 - Reliable dependency management
 - Script execution and build processes
 - Security audit capabilities
- **Language:** TypeScript
 - Static typing superset of JavaScript
 - Enhanced developer experience with better tooling
 - Compile-time error detection
- **Code Quality:** ESLint with Next.js configuration
 - Consistent code style enforcement
 - Best practice recommendations
 - Integration with development workflow
- **Database Management:**
 - **Drizzle Kit:** Migration generation and application
 - **Custom Scripts:** Database seeding, resetting, and cleaning
 - **Environment-specific:** Separate configurations for development and production
- **Testing:** @playwright/test
 - End-to-end testing framework
 - Cross-browser testing capabilities
 - Visual regression testing support
- **Environment Configuration:** dotenv
 - Secure environment variable management
 - Separation of configuration from code
 - Multi-environment support

6. Additional Libraries & Utilities

UI & Utility Libraries:

- **clsx:** Conditional className joining for dynamic styling
- **date-fns:** Comprehensive date manipulation and formatting
- **lucide-react:** Beautiful and consistent icon library
- **sonner:** Toast notifications for user feedback
- **recharts:** Data visualization and charting components

Database Drivers:

- **better-sqlite3:** Local development database support
- **postgres:** Production PostgreSQL driver with connection pooling

Validation & Schema:

- **zod**: Runtime type checking and schema validation
- **TypeScript**: Compile-time type safety

Development Tools:

- **tsx**: TypeScript execution for build scripts
- **Custom Scripts**: Database management and authentication utilities

Project Architecture

Directory Structure

rail-madad/	
├─ app/	# Next.js application directory
│ └─ api/	# Backend API endpoints
│ └─ auth/	# Authentication APIs
│ └─ complaints/	# Complaint management APIs
│ └─ users/	# User management APIs
│ └─ dashboard/	# Dashboard data APIs
│ └─ auth/	# Authentication UI pages
│ └─ complaints/	# Complaint management pages
│ └─ dashboard/	# Role-based dashboard pages
│ └─ globals.css	# Global styles
│ └─ layout.tsx	# Root layout component
│ └─ page.tsx	# Homepage component
├─ components/	# Reusable React UI components
│ └─ ui/	# Base UI components (ShadCN)
│ └─ forms/	# Form components
│ └─ tables/	# Data table components
│ └─ dashboard/	# Dashboard-specific components
├─ db/	# Database configuration and schema
│ └─ schema.ts	# Database schema definitions
│ └─ migrations/	# Database migration files
│ └─ seed.ts	# Database seeding scripts
├─ lib/	# Utility functions and configurations
│ └─ auth.ts	# Authentication utilities
│ └─ db.ts	# Database connection setup
│ └─ jwt.ts	# JWT token management
│ └─ ai.ts	# AI integration utilities
│ └─ utils.ts	# General utility functions
├─ public/	# Static assets
│ └─ images/	# Application images
│ └─ icons/	# Application icons
├─ scripts/	# Build and maintenance scripts
│ └─ db-reset.ts	# Database reset script
│ └─ auth-clear.ts	# Authentication clearing script
│ └─ seed-data.ts	# Data seeding script
├─ types/	# TypeScript type definitions
├─ .env.example	# Environment variables template
├─ .env.local	# Local environment variables
├─ package.json	# Project dependencies and scripts
├─ tailwind.config.js	# TailwindCSS configuration
├─ next.config.js	# Next.js configuration
├─ tsconfig.json	# TypeScript configuration
└─ README.md	# Project documentation

Data Flow Architecture

1. **Frontend Layer**: Next.js React components with TypeScript
2. **API Layer**: Next.js API routes handling business logic
3. **Authentication Layer**: JWT-based auth with role-based access
4. **Database Layer**: PostgreSQL with Drizzle ORM
5. **AI Layer**: Google Gemini AI for intelligent processing

Development Workflow

Environment Setup

```
# Clone repository
git clone <repository-url>
cd rail-madad

# Install dependencies
npm install

# Setup environment variables
cp .env.example .env
# Configure DATABASE_URL, JWT_SECRET, GEMINI_API_KEY, etc.

# Database setup
npm run db:generate # Generate migrations
npm run db:migrate  # Apply migrations
npm run db:seed      # Seed initial data

# Start development server
npm run dev
```

Available Scripts

- **Development:**
 - `npm run dev` - Start development with auth clearing
 - `npm run dev:no-clear` - Start development without auth clearing
 - `npm run build` - Build for production
 - `npm run start` - Start production server
- **Database Management**
 - `npm run db:generate` - Generate new migrations
 - `npm run db:migrate` - Apply pending migrations
 - `npm run db:seed` - Seed database with initial data
 - `npm run db:reset` - Reset database (drop and recreate)
 - `npm run db:clean` - Clean database (remove all data)
- **Utilities:**
 - `npm run auth:clear` - Clear authentication cookies
 - `npm run type-check` - Run TypeScript compiler checks
 - `npm run lint` - Run ESLint code quality checks

Key Features & Capabilities

Core Functionality

- **AI-Powered Complaint Management:** Automatic categorization and priority assignment
- **Role-Based Access Control:** Separate interfaces for passengers, staff, and administrators
- **Real-Time Tracking:** Live updates on complaint status and resolution
- **File Attachments:** Support for images and documents in complaints
- **Responsive Design:** Mobile-first approach with cross-device compatibility

Advanced Features

- **Sentiment Analysis:** AI-driven emotion detection in complaints
- **Automated Routing:** Intelligent assignment of complaints to appropriate departments
- **Performance Analytics:** Comprehensive reporting and dashboard insights
- **Audit Logging:** Complete tracking of all system actions and changes
- **Multi-Language Support:** Prepared for localization and internationalization

Security & Performance

- **Enterprise-Grade Security:** JWT authentication with secure session management
- **Scalable Architecture:** Cloud-native design with horizontal scaling capabilities
- **Performance Optimization:** Server-side rendering and static generation
- **Error Handling:** Comprehensive error tracking and user-friendly error messages

Deployment & Production

Production Stack

- **Hosting:** Vercel (recommended) or any Node.js-compatible platform
- **Database:** Neon PostgreSQL for production workloads
- **CDN:** Built-in Vercel CDN for static assets
- **Monitoring:** Built-in Next.js analytics and error reporting

Environment Configuration

- **Development:** Local database with hot reloading
- **Staging:** Mirror of production with test data
- **Production:** Optimized builds with performance monitoring

Technology Rationale

Why Next.js 14?

- **Full-Stack Framework:** Combines frontend and backend in a single codebase
- **App Router:** Modern routing with improved performance and developer experience
- **Built-in Optimization:** Image optimization, font loading, and bundle splitting
- **Deployment Ready:** Seamless deployment to Vercel and other platforms

Why TypeScript?

- **Type Safety:** Catch errors at compile time rather than runtime
- **Better Developer Experience:** Enhanced IDE support and refactoring capabilities
- **Team Collaboration:** Clear interfaces and documentation through types
- **Maintainability:** Easier to maintain and scale large codebases

Why Drizzle ORM?

- **Type Safety:** Full TypeScript integration with database schemas
- **Performance:** Lightweight with minimal overhead
- **Migration Support:** Robust schema evolution and version control
- **SQL Flexibility:** Direct SQL access when needed

Why Google Gemini AI?

- **Advanced Capabilities:** State-of-the-art language understanding
- **API Stability:** Enterprise-grade reliability and support
- **Cost Effectiveness:** Competitive pricing for AI capabilities
- **Integration:** Well-documented APIs with excellent tooling

Future Enhancements

Planned Features

- **Mobile Application:** Native iOS and Android apps
- **Advanced Analytics:** Machine learning insights and predictions
- **Integration APIs:** Third-party system integrations
- **Multi-Tenant Support:** Support for multiple railway zones

Technical Improvements

- **Microservices Architecture:** Gradual migration to microservices
- **Real-Time Features:** WebSocket integration for live updates
- **Advanced Caching:** Redis integration for improved performance
- **API Rate Limiting:** Advanced rate limiting and throttling

Conclusion

The Rail Madad AI Complaint Management System represents a modern, scalable solution built with cutting-edge technologies. The chosen tech stack provides:

- **Reliability:** Battle-tested technologies with strong community support
- **Scalability:** Architecture designed to handle growing user demands
- **Maintainability:** Clean code practices and comprehensive documentation
- **Security:** Enterprise-grade security measures and best practices
- **Performance:** Optimized for speed and user experience
- **Innovation:** AI integration for intelligent automation and insights

This technical foundation ensures the system can effectively serve the Indian Railways community while providing a platform for continuous improvement and expansion.